
Water Quality Hazard Prediction: End-to-End MLOps Pipeline

A Project Report Submitted in Partial Fulfilment of the Requirements for
BSc (Hons) Computer Science with Artificial Intelligence

Student Name: Abiska Sharma
Student ID: 25123780
Instructor: Umesh Regmi
Date of Submission: 2nd October 2026

Contents

1	Introduction	3
1.1	Background	3
1.2	Problem Statement	3
1.3	Purpose of the Project	4
2	Objectives	5
3	Methodology	6
3.1	Tools and Frameworks	6
3.2	Dataset Description	6
3.3	Feature Engineering and Selection	7
3.4	Model Training and Hyperparameter Tuning	8
3.5	Experiment Tracking and Model Versioning	11
3.6	Workflow Automation	11
3.7	Drift Monitoring	12
4	Project Implementation	13
4.1	System Architecture and Data Flow	13
4.2	Training and Model Export	14
4.3	FastAPI Microservice Implementation	15
4.4	Containerisation with Docker	15
4.5	Django Asynchronous View Implementation	16
4.6	Airflow Retraining DAG	17
4.7	Drift Monitoring with Evidently AI	17
5	Results and Output	19
5.1	Model Performance Evaluation	19
5.1.1	Regression: Next-Day pH Forecast	19
5.1.2	Classification: Hazard Flag	19
5.1.3	Drift Monitoring Results	20
5.2	MLOps Artefacts and Interfaces	20
6	Challenges and Limitations	24

6.1	Technical Hurdles	24
6.2	System Constraints	25
7	Conclusion	26
8	Future Scope	27
9	References	28
10	Appendices	30
	Appendix A: Terminal Setup and Deployment Commands	30

Chapter 1

Introduction

1.1 Background

The capacity to identify unsafe conditions in water has been a vital component of any contemporary environmental monitoring system, and potable water is one of the most fundamental public health concerns. Most prototypes of the software to automate this task terminate in the training of a model in a notebook, as is used in traditional water quality assessment. Such prototypes are rarely deployable: there is no versioning of the model, no retraining when new data comes in, it is tightly coupled with whatever application is using it, and no one's looking to see if the data coming in is beginning to look different than the data that the model was trained on.

This project, Water Quality Hazard Prediction: End-to-End MLOps Pipeline, addresses that gap. It uses real daily sensor readings collected by the United States Geological Survey (USGS) across 36–37 water monitoring sites in Georgia, USA, to build two models from eight normalised sensor features: a regression model that forecasts next-day pH as a continuous value, and a binary classification model that flags whether a reading should be treated as a statistical hazard. Around these models, the project builds a complete machine learning operations (MLOps) lifecycle: hyperparameter tuning with Optuna, experiment tracking with MLflow, automated retraining orchestration with Apache Airflow, data drift monitoring with Evidently AI, containerised deployment with Docker, and a Django web portal backed by SQLite for end users.

1.2 Problem Statement

Conventional implementations of predictive water-quality systems suffer from the following interrelated deficiencies, each of which this project directly addresses:

- (a) **Tight Coupling and Latency.** When the presentation layer and the model inference logic live in the same process, every prediction blocks the web server, and

the entire web application must be redeployed whenever the model changes.

- (b) **Lack of Boundary Type Safety.** Without a validation contract at the service boundary, malformed or out-of-range sensor readings can silently reach the model and produce misleading predictions rather than being rejected outright.
- (c) **Manual, Untracked Model Development.** When hyperparameters are tuned by hand and results are only visible as printed output in a notebook, there is no durable record of which configuration produced which model, making results difficult to reproduce or compare.
- (d) **Silent Model Degradation.** A model trained on historical data can lose reliability as the distribution of incoming data shifts over time. Without active monitoring, this degradation goes unnoticed.

1.3 Purpose of the Project

The purpose of this project is to design and deploy a decoupled, automated, and monitored machine learning system that addresses the four problems above. The system consists of the following cooperating components:

- A **Django MVT web portal**, backed by SQLite, responsible for form-based sensor-reading capture, asynchronous communication with the inference service, storage of every submitted reading and its prediction, and presentation of results and a prediction history.
- A **FastAPI machine learning microservice**, packaged as a Docker container, responsible exclusively for validating readings with Pydantic, running inference with the trained scikit-learn pipelines, and returning a structured prediction. This is the only part of the system that touches the trained model directly.
- A **Jupyter notebook** covering exploratory data analysis, feature selection, model training, evaluation, and hyperparameter tuning with Optuna, with every run logged to MLflow.
- An **Apache Airflow DAG** that orchestrates the data-reshaping and model-training scripts on demand, so retraining does not depend on manually re-running scripts.
- An **Evidently AI drift-monitoring step** that statistically compares a current data sample against the training distribution and reports which features have drifted.

Django and FastAPI communicate over HTTP using `httpx.AsyncClient()`, an asynchronous client library, so that the Django web server is never blocked while waiting for a prediction. Every payload crossing this boundary is validated twice: once by a Django form, and again by a Pydantic schema on the FastAPI side.

Chapter 2

Objectives

The project is guided by the following core objectives:

1. **Build a dual-purpose machine learning pipeline** on real USGS sensor data, training both a **regression model** (next-day pH forecasting) and a **binary classification model** (hazard flagging) from the same eight engineered features, using Python and Scikit-learn.
2. **Perform feature engineering and selection**, including reshaping the raw spatio-temporal dataset into a tidy table and removing highly collinear features identified through correlation analysis.
3. **Optimise the regression model's hyperparameters with Optuna**, running a study of 15 trials with 3-fold cross-validation on the training split as the optimisation objective.
4. **Track every tuning run in MLflow**, so that results are reproducible and comparable across runs.
5. **Automate the retraining workflow with Apache Airflow**, orchestrating the data-reshaping and model-training scripts as a directed acyclic graph (DAG) that can be triggered on demand.
6. **Deploy the trained models as a containerised REST API** using FastAPI and Docker, exposing a documented `/predict` endpoint whose inputs are validated by a Pydantic `SensorReading` schema.
7. **Monitor data drift with Evidently AI**, statistically comparing a current data sample against the training distribution and reporting which features have drifted and by how much.
8. **Integrate a Django web portal** with SQLite for user-facing sample submission, asynchronous inter-service communication via `httpx.AsyncClient()`, and a results and history dashboard.

Chapter 3

Methodology

3.1 Tools and Frameworks

The project was implemented using a curated stack of Python-based tools, each selected for a specific responsibility, as summarised in Table 3.1.

Table 3.1: Technology Stack and Responsibilities

Component	Responsibility
Python 3.12	Core programming language for all components
Pandas / NumPy	Data reshaping, cleaning, and numerical processing
SciPy	Loading the raw <code>.mat</code> spatio-temporal dataset
Scikit-learn	Random Forest regressor, Gradient Boosting classifier
Optuna	Hyperparameter optimisation (15 trials)
MLflow	Experiment tracking (parameters, metrics, model artefact)
Apache Airflow	Orchestration of the retraining workflow (LocalExecutor)
FastAPI	REST inference microservice
Pydantic	Declarative data validation at the service boundary
Docker / Docker Compose	Containerisation of FastAPI and Django services
Evidently AI	Data-drift detection and HTML drift reports
Django (MVT, async views)	Web portal: forms, templates, prediction history
SQLite	Relational database for the Django application
httpx	Asynchronous HTTP client for Django-to-FastAPI calls
Joblib	Serialisation of the trained model pipelines

3.2 Dataset Description

The dataset used is the *Water Quality Prediction* dataset published on the UCI Machine Learning Repository (dataset #733), originally collected by the United States Geological

Survey across 36–37 water monitoring sites in Georgia, USA, and associated with a 2019 publication by Zhao, Gkountouna, and Pfoser on spatial auto-regressive dependency learning (Zhao et al., 2019).

The raw data is supplied as a MATLAB `.mat` file structured as a spatio-temporal cell array rather than a flat table: separate training and test arrays (`X_tr`, `X_te`) each containing one 37×11 matrix per day (37 sites, 11 raw sensor features), alongside corresponding target-pH arrays (`Y_tr`, `Y_te`). The split between training and test data is **temporal**, **not random**: the training array covers 423 contiguous days, and the test array covers the following 282 contiguous days.

This raw structure was reshaped into one tidy row per (site, day) observation, producing 26,085 total rows — 15,651 in the training split and 10,434 in the test split. The 11 raw features are summarised in Table 3.2.

Table 3.2: Raw Dataset Features (11 total, before feature selection)

Feature	Description
conductance_max/min/mean	Specific conductance ($\mu\text{S}/\text{cm}$), daily max/min/mean
ph_max/min	Field pH, daily maximum and minimum
do_max/mean/min	Dissolved oxygen (mg/L), daily max/mean/min
temp_max/mean/min	Water temperature ($^{\circ}\text{C}$), daily max/mean/min

Important data limitation. All feature and target values in this dataset are min–max normalised to the range $[0, 1]$, and the original scaler used to produce this normalisation is not provided. This means the true pH scale cannot be recovered from this file alone, which has direct consequences for how the hazard label is defined (Section 3.3).

```
mat = sio.loadmat("../dataset/water_dataset.mat")
print("Top-level keys:", [k for k in mat.keys() if not k.startswith("_")])
print("features shape:", mat["features"].shape)
print("X_tr shape (days):", mat["X_tr"].shape, "X_te shape (days):", mat["X_te"].shape)
print("Y_tr shape:", mat["Y_tr"].shape, "Y_te shape:", mat["Y_te"].shape)

Top-level keys: ['X_tr', 'X_te', 'Y_tr', 'Y_te', 'location_group', 'features', 'location_ids']
features shape: (1, 11)
X_tr shape (days): (1, 423) X_te shape (days): (1, 282)
Y_tr shape: (37, 423) Y_te shape: (37, 282)
```

Figure 3.1: Loading the raw `.mat` file with `scipy.io.loadmat()`.

```
df = pd.concat(
    [buildmat("X_tr"), mat["X_tr"], "train"], buildmat("Y_tr"), mat["Y_tr"], "test"),
    ignore_index=True,
)
df["group"] = df["site_num"].map(group_labels)
return df

df = reshape_to_long(mat)
print("Tidy dataset shape:", df.shape)
df.head()
```

Tidy dataset shape: (26085, 18)

day_id	site_id	site_num	location_id	split	conductance_max	ph_max	ph_min	conductance_min	conductance_mean	do_max	do_mean	do_min	temp_m
0	0	1	219840	train	0.001131	0.804615	0.001120	0.001113	0.677632	0.641463	0.765152	0.787402	0.292
1	0	1	219820	train	0.001170	0.871795	0.001159	0.001152	0.703947	0.829268	0.772727	0.795276	0.292
2	0	2	219850	train	0.001326	0.884615	0.001198	0.001220	0.677632	0.829268	0.750000	0.755906	0.300
3	0	3	220303	train	0.014094	0.858974	0.001238	0.003926	0.697268	0.829268	0.772727	0.771854	0.294
4	0	4	220303	train	0.088109	0.858974	0.010768	0.029297	0.684211	0.829268	0.765152	0.755906	0.294

Figure 3.2: Tidy dataset after reshaping (one row per site and day).

3.3 Feature Engineering and Selection

Exploratory analysis of the 11 raw features showed that three of them were almost perfectly correlated with features already being kept: `temp_min` and `temp_max` with `temp_mean` (correlation ≈ 0.98 – 0.99), and `do_min` with `do_mean` (correlation ≈ 0.97). These three features were dropped as redundant, collinear inputs. The remaining **eight fea-**

tures were used for both models: `conductance_max`, `conductance_min`, `conductance_mean`, `ph_max`, `ph_min`, `do_max`, `do_mean`, `temp_mean`.

Because the dataset's normalisation scaler is unrecoverable, there is no pre-existing hazard label in the raw data. A binary hazard label was therefore derived using the classic **interquartile-range (IQR) outlier rule**, computed on the *training split's target pH only* (to avoid leaking test-set information into the label definition): any reading whose pH falls more than $1.5 \times \text{IQR}$ outside the training interquartile range is flagged as a hazard. This produced a safe band of normalised pH between **0.6065 and 0.7176**, with approximately **4.6%** of readings falling outside it and being labelled as hazards. **This label is a statistical proxy, not a regulatory safety threshold** such as the EPA's pH 6.5–8.5 potable-water range. This limitation is acknowledged explicitly and discussed further in Chapter 6.

The data-cleaning pipeline, implemented in `data_pipeline.py`, follows these steps:

1. Load the raw `.mat` file using `scipy.io.loadmat()`.
2. Reshape the spatio-temporal cell arrays into one tidy row per (site, day).
3. Drop the three collinear features identified above.
4. Derive the binary hazard label via the IQR rule on the training split.
5. Export the tidy dataset as a CSV for use by the notebook, the training script, and the drift-monitoring step.

3.4 Model Training and Hyperparameter Tuning

Two models were trained from the same eight features, reflecting the project's dual regression-and-classification design:

- A **Random Forest Regressor** (`sklearn.ensemble.RandomForestRegressor`) predicting next-day pH as a continuous value.
- A **Gradient Boosting Classifier** (`sklearn.ensemble.GradientBoostingClassifier`) predicting the binary hazard flag.

Both models were wrapped in an `sklearn.Pipeline` with median imputation followed by standard scaling, so that the exact training-time preprocessing is reused unchanged at inference time.

The regressor's hyperparameters were optimised using **Optuna**. A single study of 15 trials searched over the number of trees (`n_estimators`), maximum tree depth (`max_depth`), and minimum samples per leaf (`min_samples_leaf`), using 3-fold cross-validation on the training split with negative root-mean-squared-error as the objective. The best configuration found was `n_estimators=300`, `max_depth=16`, `min_samples_leaf=5`.

```
[1]: import json
import time

import joblib
import numpy as np
import pandas as pd
import scipy.io as sio
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.ensemble import GradientBoostingClassifier, RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
    mean_squared_error, mean_absolute_error, r2_score,
    confusion_matrix, RocCurveDisplay,
)

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

sns.set_theme(style="whitegrid")
RANDOM_STATE = 42
```

Figure 3.3: Library imports (Pandas, SciPy, Scikit-learn, Seaborn).

```
# Human-readable aliases for the 11 raw USGS feature names
FEATURE_ALIASES = {
    "Specific conductance, water, unfiltered, microsiemens per centimeter at 25 degrees Celsius (Maximum)": "conductance_max",
    "pH, water, unfiltered, field, standard units (Maximum)": "ph_max",
    "pH, water, unfiltered, field, standard units (Minimum)": "ph_min",
    "Specific conductance, water, unfiltered, microsiemens per centimeter at 25 degrees Celsius (Minimum)": "conductance_min",
    "Specific conductance, water, unfiltered, microsiemens per centimeter at 25 degrees Celsius (Mean)": "conductance_mean",
    "Dissolved oxygen, water, unfiltered, milligrams per liter (Maximum)": "do_max",
    "Dissolved oxygen, water, unfiltered, milligrams per liter (Mean)": "do_mean",
    "Dissolved oxygen, water, unfiltered, milligrams per liter (Minimum)": "do_min",
    "Temperature, water, degrees Celsius (Mean)": "temp_mean",
    "Temperature, water, degrees Celsius (Minimum)": "temp_min",
    "Temperature, water, degrees Celsius (Maximum)": "temp_max",
}

TARGET_COL = "target_ph"

def reshape_to_long(mat):
    """Convert the .mat cell-array format into one tidy long-format DataFrame,
    one row per (site, day)."""
    feature_names = [f[i] for f in mat["features"]][0]
    location_ids = mat["location_ids"].ravel()

    group_labels = []
    for gi, arr in enumerate(mat["location_group"])[0]:
        for loc in arr.ravel():
            group_labels.append(loc)

    def build_x, y, split_name:
        n_days = X.shape[1]
        rows = []
        for t in range(n_days):
            day_matrix = X[0, t] # (n_sites, 11)
            for s in range(day_matrix.shape[0]):
                row = [
                    "day_idx": t, "site_idx": s, "site_num": s + 1,
                    "location_id": int(location_ids[s]), "split": split_name,
                ]
                for f_idx, fname in enumerate(feature_names):
                    row[FEATURE_ALIASES[fname]] = day_matrix[s, f_idx]
                row[TARGET_COL] = Y[s, t]
                rows.append(row)
        return pd.DataFrame(rows)
```

Figure 3.4: Reshaping the .mat cell arrays into a tidy table.

```
[4]: print("Missing values per column:")
print(df.isnull().sum())
print("\nSplit sizes:")
print(df["split"].value_counts())
df.describe().T
```

Missing values per column:

column	count	mean	std	min	25%	50%	75%	max
day_idx	26085.0	1.828000e+02	1.130952e+02	0.000000e+00	8.800000e+01	1.760000e+02	2.640000e+02	422.0
site_idx	26085.0	1.800000e+01	1.067728e+01	0.000000e+00	9.000000e+00	1.800000e+01	2.700000e+01	36.0
site_num	26085.0	1.900000e+01	1.067728e+01	1.000000e+00	1.000000e+01	1.900000e+01	2.800000e+01	37.0
location_id	26085.0	1.138139e+07	3.552055e+07	2.198840e+06	2.203900e+06	2.336152e+06	2.337170e+06	219897945.0
conductance_max	26085.0	7.164787e+02	1.684075e+01	5.263150e-04	1.910331e-03	2.631579e-03	5.497076e-03	1.0
ph_max	26085.0	8.873486e-01	3.538604e-02	4.102564e-01	8.717949e-01	8.846154e-01	9.102564e-01	1.0
ph_min	26085.0	3.098628e-02	1.222393e-01	2.554020e-04	1.591356e-03	2.220039e-03	3.713163e-03	1.0
conductance_min	26085.0	4.707879e-02	1.366863e-01	3.078125e-04	1.777344e-03	2.421875e-03	4.902344e-03	1.0
conductance_mean	26085.0	5.509415e-01	1.196887e-01	1.184211e-01	4.802632e-01	5.460526e-01	6.315789e-01	1.0
do_max	26085.0	8.572369e-01	3.116706e-02	7.195122e-01	8.414634e-01	8.536585e-01	8.780488e-01	1.0
do_mean	26085.0	5.851892e-01	1.463075e-01	6.818182e-02	5.075758e-01	5.833333e-01	6.893939e-01	1.0
do_min	26085.0	5.573587e-01	1.728360e-01	3.149606e-02	4.724409e-01	5.669291e-01	6.771654e-01	1.0
temp_mean	26085.0	5.881742e-01	2.004031e-01	5.947400e-02	4.718740e-01	6.000000e-01	7.511740e-01	1.0

Split sizes:

split	count
train	15651
test	10434

Name: count, dtype: int64

Figure 3.5: Missing-value check and summary statistics.

```
[5]: fig, ax = plt.subplots(figsize=(6, 4))
sns.histplot(df[TARGET_COL], bins=40, kde=True, ax=ax, color="#1f6f5c")
ax.set_title("Distribution of target pH (normalized)")
ax.set_xlabel("Normalized pH")
plt.tight_layout()
plt.show()
```

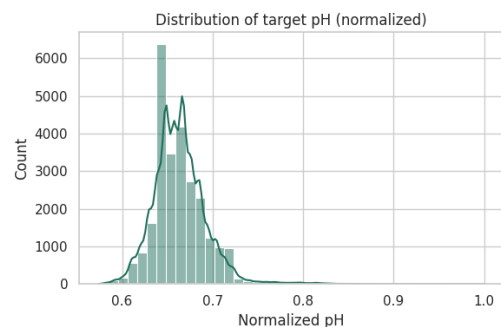


Figure 3.6: Distribution of the (normalised) target pH.

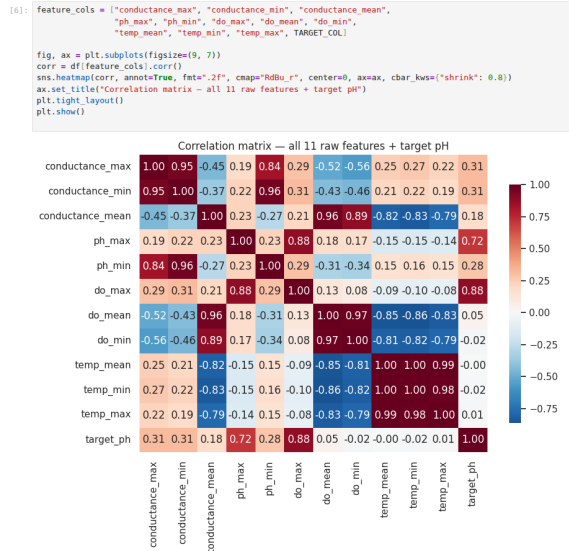


Figure 3.7: Correlation matrix of all 11 raw features and the target.



Figure 3.8: The eight selected model features and the IQR-based hazard label.

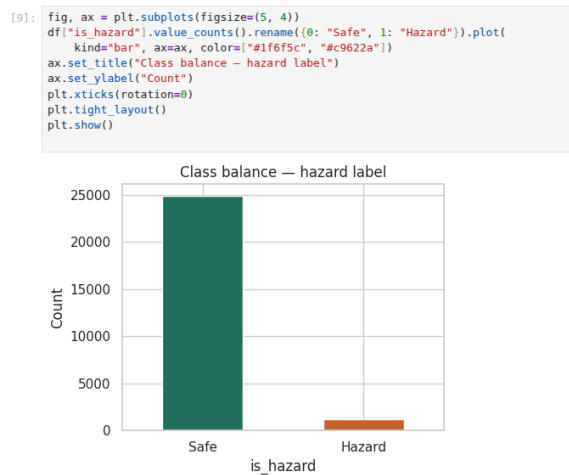


Figure 3.9: Class balance of the hazard label.

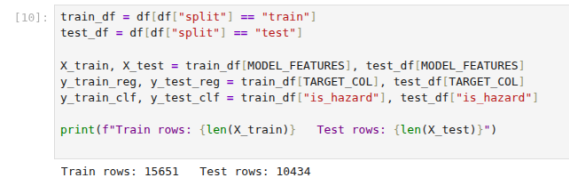


Figure 3.10: Temporal train/test split (15,651 / 10,434 rows).

```
[11]: def make_regression_pipeline():
    return Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("model", RandomForestRegressor(
            n_estimators=300, max_depth=12, min_samples_leaf=3,
            random_state=RANDOM_STATE, n_jobs=-1)),
    ])

def make_classification_pipeline():
    return Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("model", GradientBoostingClassifier(
            n_estimators=250, max_depth=3, learning_rate=0.05, random_state=RANDOM_STATE)),
    ])

reg_pipe = make_regression_pipeline()
clf_pipe = make_classification_pipeline()

[12]: t0 = time.time()
reg_pipe.fit(X_train, y_train_reg)
print(f'Regressor trained in {(time.time() - t0):.2f}s')

t0 = time.time()
clf_pipe.fit(X_train, y_train_clf)
print(f'Classifier trained in {(time.time() - t0):.2f}s')

Regressor trained in 1.16s
Classifier trained in 3.85s
```

Figure 3.11: Regression and classification pipeline definitions.

```
[19]: import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)
from sklearn.model_selection import cross_val_score

def objective(trial):
    n_estimators = trial.suggest_int("n_estimators", 100, 350, step=50)
    max_depth = trial.suggest_int("max_depth", 6, 20)
    min_samples_leaf = trial.suggest_int("min_samples_leaf", 1, 6)

    pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("model", RandomForestRegressor(
            n_estimators=n_estimators, max_depth=max_depth,
            min_samples_leaf=min_samples_leaf, random_state=RANDOM_STATE, n_jobs=-1)),
    ])

    score = cross_val_score(pipe, X_train, y_train_reg, cv=3,
                           scoring="neg_root_mean_squared_error", n_jobs=1)

    return -score.mean()

study = optuna.create_study(direction="minimize", study_name="ph_regressor_tuning")
study.optimize(objective, n_trials=15)

print('Best params:', study.best_params)
print('Best CV RMSE:', study.best_value)

Best params: {'n_estimators': 350, 'max_depth': 20, 'min_samples_leaf': 4}
Best CV RMSE: 0.012243431477976829
```

Figure 3.12: Optuna objective function and 15-trial study.

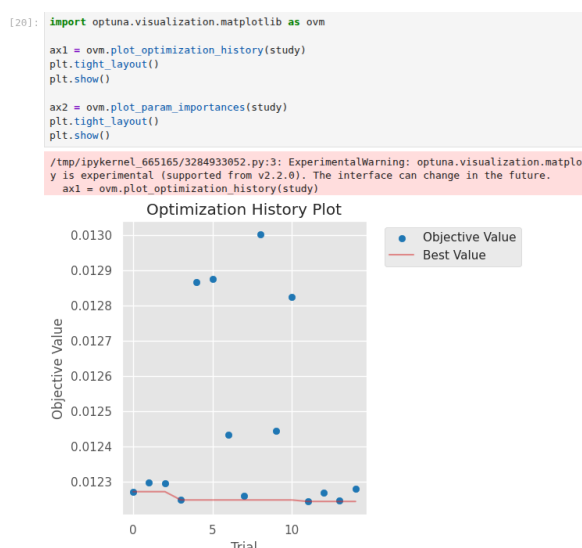


Figure 3.13: Optuna optimisation history.

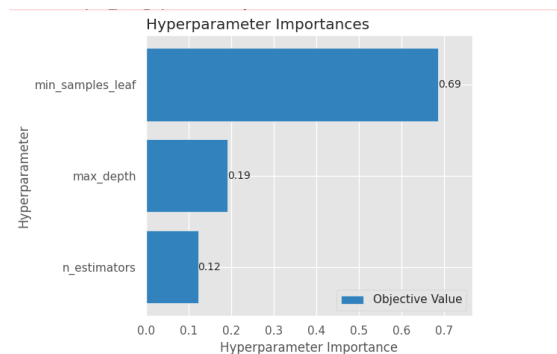


Figure 3.14: Optuna hyperparameter importances.

3.5 Experiment Tracking and Model Versioning

MLflow was used to record, for the tuned training run, the hyperparameters found by Optuna, the resulting test-set metrics (RMSE, MAE, R^2), and the trained pipeline itself as a logged model artefact, tracked against a local SQLite backend store. This provides a durable, inspectable record of the tuning result rather than relying on console output alone.

3.6 Workflow Automation

Apache Airflow automates the retraining workflow through a DAG named `water_quality_training_pipeline`, consisting of two sequential tasks implemented as

BashOperators: `reshape_data`, which runs `data_pipeline.py` to regenerate the tidy dataset from the raw `.mat` file, and `train_models`, which runs `train.py` to retrain both models and overwrite the serialised model artefacts consumed by the FastAPI service. The DAG is configured with `schedule=None`, meaning it is triggered manually rather than on a fixed schedule — modelling a realistic “retrain on demand” workflow for a project evaluated once, rather than a system that needs to run nightly. The DAG is described further in Section 4.6.

3.7 Drift Monitoring

Evidently AI compares a **reference** dataset against a **current** dataset and reports, for each feature, a statistical drift score against a fixed threshold. In this project, the training split is used as the reference distribution and the test split is used as the current distribution. Because the dataset’s train/test split is **temporal** rather than random, this comparison is not an artificial simulation of drift: it is a genuine test of whether the later monitoring period’s sensor readings differ statistically from the earlier training period’s readings. The drift check uses the Wasserstein distance (normalised) with a threshold of 0.1 per feature, and is implemented as a standalone script producing both a CSV summary and an HTML report. Results are discussed in Section 5.1.3.

Chapter 4

Project Implementation

4.1 System Architecture and Data Flow

The system has two cooperating flows, shown in Figure 4.1. The **training flow** is orchestrated by Airflow and produces the serialised model artefacts consumed by the inference service. The **serving flow** carries a user's request from the browser through Django and the containerised FastAPI service to the model and back.

FIGURE 4.1: SYSTEM ARCHITECTURE: TRAINING FLOW & SERVING FLOW

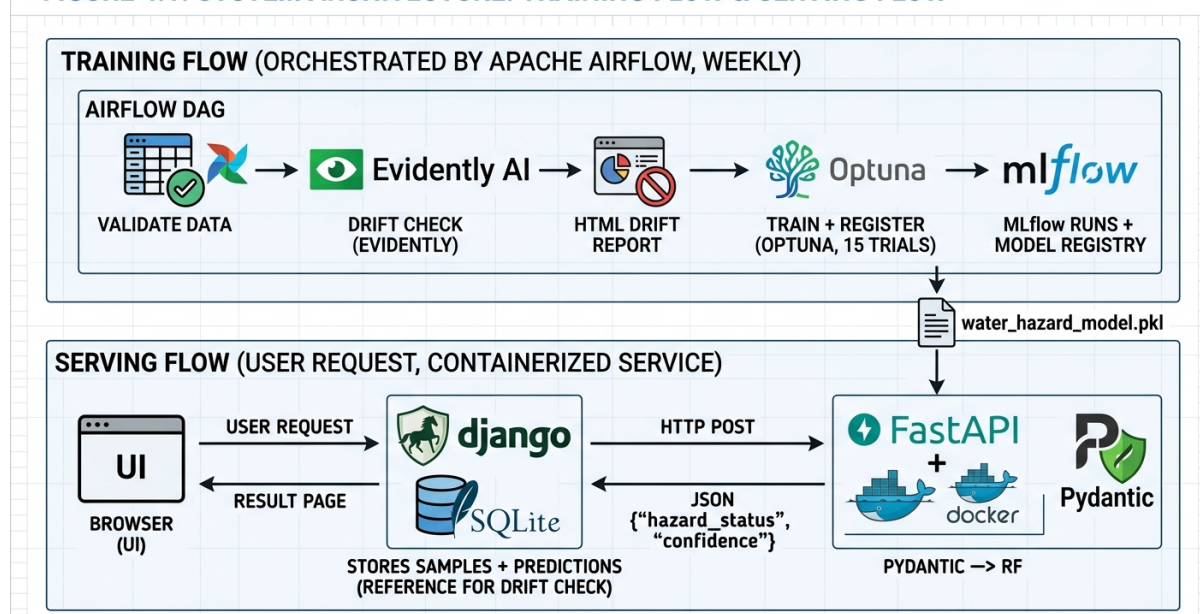


Figure 4.1: System Architecture: Training Flow (Airflow, Optuna, MLflow, Evidently) and Serving Flow (Django, FastAPI, Docker)

This architecture separates concerns cleanly: Django never performs inference, FastAPI never renders HTML or manages sessions, and the training pipeline is independent of both serving components. The services communicate only through a versionable JSON contract and shared model artefacts.

4.2 Training and Model Export

The notebook trains both pipelines, evaluates them on the held-out test split, logs the tuned regressor to MLflow (Figures 4.3–4.3), and exports the trained pipelines (`ph_regressor.joblib` and `hazard_classifier.joblib`) together with a metadata file for the API (Figure 4.4).

```
[21]: best_params = study.best_params

tuned_reg_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", RandomForestRegressor(
        n_estimators=best_params["n_estimators"],
        max_depth=best_params["max_depth"],
        min_samples_leaf=best_params["min_samples_leaf"],
        random_state=RANDOM_STATE, n_jobs=-1)),
])

tuned_reg_pipe.fit(X_train, y_train_reg)
y_pred_tuned = tuned_reg_pipe.predict(X_test)

tuned_metrics = {
    "rmse": float(np.sqrt(mean_squared_error(y_test_reg, y_pred_tuned))),
    "mae": float(mean_absolute_error(y_test_reg, y_pred_tuned)),
    "r2": float(r2_score(y_test_reg, y_pred_tuned)),
}

print("BEFORE tuning (original hand-picked settings):")
for k, v in reg_metrics.items():
    print(f" {k.upper():5s}: {v:.5f}")

print("\nAFTER tuning (Optuna best params):")
for k, v in tuned_metrics.items():
    print(f" {k.upper():5s}: {v:.5f}")

BEFORE tuning (original hand-picked settings):
RMSE : 0.01160
MAE   : 0.00661
R2    : 0.84483

AFTER tuning (Optuna best params):
RMSE : 0.01157
MAE   : 0.00660
R2    : 0.84566
```

Figure 4.2: Metrics before and after Optuna tuning.

```
[22]: import mlflow
import mlflow.sklearn

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("ph_regressor")

with mlflow.start_run(run_name="rf_regressor_tuned"):
    mlflow.log_params(best_params)
    mlflow.log_metrics(tuned_metrics)
    mlflow.sklearn.log_model(
        tuned_reg_pipe, name="model", serialization_format="cloudpickle"
    )
    print("Run logged to MLflow.")

2026/10/02 18:49:54 WARNING mlflow.sklearn: Saving scikit-learn models in the
old format. These formats rely on Python's object serialization mechanism, which ca
ould be unsafe in some environments. A safer alternative is the 'skops' format. For more information, see: https://
mlflow.org/docs/latest/python_api/mlflow.sklearn.html#mlflow-sklearn-log-model
Run logged to MLflow.
```

Figure 4.3: Logging the tuned regressor to MLflow.

```
[18]: import os
os.makedirs("../ml_pipeline/models", exist_ok=True)

joblib.dump(reg_pipe, "../ml_pipeline/models/ph_regressor.joblib")
joblib.dump(clf_pipe, "../ml_pipeline/models/hazard_classifier.joblib")

metadata = {
    "model_features": MODEL_FEATURES,
    "target_col": TARGET_COL,
    "hazard_col": "is_hazard",
    "hazard_bounds_normalized_ph": list(hazard_bounds),
    "feature_importances_regressor": importances.to_dict(),
    "regression_metrics": reg_metrics,
    "classification_metrics": clf_metrics,
    "train_rows": int(len(X_train)),
    "test_rows": int(len(X_test)),
    "trained_at_unix": time.time(),
    "sklearn_pipeline_steps": ["imputer(median)", "scaler(standard)", "model"],
}

with open("../ml_pipeline/models/metadata.json", "w") as f:
    json.dump(metadata, f, indent=2)

df.to_csv("../dataset/water_quality_processed.csv", index=False)
df.to_csv("../ml_pipeline/data/water_quality_processed.csv", index=False)

print("Saved ../ml_pipeline/models/ph_regressor.joblib, hazard_classifier.joblib, metadata.json")
print("Saved ../dataset/water_quality_processed.csv and ../ml_pipeline/data/water_quality_processed.csv")

Saved ../ml_pipeline/models/ph_regressor.joblib, hazard_classifier.joblib, metadata.json
Saved ../dataset/water_quality_processed.csv and ../ml_pipeline/data/water_quality_processed.csv
```

Figure 4.4: Exporting the trained pipelines and metadata with Joblib.

4.3 FastAPI Microservice Implementation

The FastAPI service (`fastapi_service.py`) loads the exported model pipelines once at start-up, defines the `SensorReading` and `HazardResponse` Pydantic schemas, and exposes the `/predict` and `/health` endpoints. Out-of-range or non-numeric inputs are rejected automatically before reaching the model.

```

1  # fastapi_service.py
2  """
3  Minimal FastAPI microservice exposing the trained hazard-prediction pipeline.
4  This is the service Django will call (e.g. via 'requests' / 'httpx') whenever
5  it needs a hazard verdict for a new set of sensor readings.
6
7  Run locally:
8  uvicorn fastapi_service:app --host 0.0.0.0 --port 8001 --reload
9
10 Example request:
11 POST http://localhost:8001/predict
12 {
13   "conductance_max": 0.05, "conductance_min": 0.05, "conductance_mean": 0.55,
14   "ph_max": 0.88, "ph_min": 0.03, "do_max": 0.85, "do_mean": 0.58, "temp_mean": 0.6
15 }
16 """
17
18 from fastapi import FastAPI, HTTPException
19 from pydantic import BaseModel, Field
20
21 from predict import predict
22
23 app = FastAPI(
24     title="Water Quality Hazard Prediction Service",
25     description="Predicts next-day pH and flags hazardous water-quality readings.",
26     version="1.0.0",
27 )
28
29
30 class SensorReading(BaseModel):
31     conductance_max: float = Field(..., ge=0, le=1)
32     conductance_min: float = Field(..., ge=0, le=1)
33     conductance_mean: float = Field(..., ge=0, le=1)
34     ph_max: float = Field(..., ge=0, le=1)
35     ph_min: float = Field(..., ge=0, le=1)
36     do_max: float = Field(..., ge=0, le=1)
37     do_mean: float = Field(..., ge=0, le=1)
38     temp_mean: float = Field(..., ge=0, le=1)
39
40
41 class HazardResponse(BaseModel):
42     predicted_ph: float
43     is_hazard: bool
44     hazard_probability: float
45     hazard_bounds_normalized_ph: dict
46
47 @app.get("/health")
48 def health():
49     return {"status": "ok"}

```

Figure 4.5: FastAPI service: module header and imports.

```

class SensorReading(BaseModel):
    conductance_max: float = Field(..., ge=0, le=1)
    conductance_min: float = Field(..., ge=0, le=1)
    conductance_mean: float = Field(..., ge=0, le=1)
    ph_max: float = Field(..., ge=0, le=1)
    ph_min: float = Field(..., ge=0, le=1)
    do_max: float = Field(..., ge=0, le=1)
    do_mean: float = Field(..., ge=0, le=1)
    temp_mean: float = Field(..., ge=0, le=1)

class HazardResponse(BaseModel):
    predicted_ph: float
    is_hazard: bool
    hazard_probability: float
    hazard_bounds_normalized_ph: dict

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict", response_model=HazardResponse)
def predict_endpoint(reading: SensorReading):
    try:
        return predict(reading.model_dump())
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))

```

Figure 4.6: Pydantic schemas and the `/predict` endpoint.

4.4 Containerisation with Docker

The FastAPI and Django services are packaged as Docker images so that they run identically on any host and start together with a single `docker compose up` command.

```

abiskababiska-IdeaPad-Slim-5-16IMH9:~/Documents/Water_Quality_Hazard_Prediction/notebook$ cd ~/Documents/Water_Quality_Hazard_Prediction
docker compose up -d
[+] up 3/3
  ✓ Container water_django_portal Running
  ✓ Container water_ml_service Started
  ✓ Container water_notebook Started
abiskababiska-IdeaPad-Slim-5-16IMH9:~/Documents/Water_Quality_Hazard_Prediction$ cd ~/Documents/Water_Quality_Hazard_Prediction/airflow
docker compose up -d
docker compose ps
[+] up 4/4
  ✓ Container water_airflow_postgres Healthy
  ✓ Container water_airflow_scheduler Running
  ✓ Container water_airflow_webserver Running
  ✓ Container water_airflow_init Exited
NAME                IMAGE                COMMAND                SERVICE        CREATED        STATUS        PORTS
water_airflow_postgres postgres:13           "docker-entrypoint.s..." postgres       29 hours ago   Up About an hour (healthy) 5432/tcp
water_airflow_scheduler water_airflow-airflow-scheduler "/usr/bin/dumb-init ..." airflow-scheduler 29 hours ago   Up About an hour (healthy) 8080/tcp
water_airflow_webserver water_airflow-airflow-webserver "/usr/bin/dumb-init ..." airflow-webserver 29 hours ago   Up About an hour (healthy) 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
abiskababiska-IdeaPad-Slim-5-16IMH9:~/Documents/Water_Quality_Hazard_Prediction/airflow$ cd ~/Documents/Water_Quality_Hazard_Prediction/ml_pipeline
source venv/bin/activate
cd ../notebook
jupyter notebook
bash: venv/bin/activate: No such file or directory
[1] 2026-10-03 18:01:47.753 ServerApp jupyter_lsp | extension was successfully linked.
[1] 2026-10-03 18:01:47.755 ServerApp jupyter_server_terminals | extension was successfully linked.
[1] 2026-10-03 18:01:47.757 ServerApp jupyterlab | extension was successfully linked.
[1] 2026-10-03 18:01:47.759 ServerApp notebook | extension was successfully linked.
[1] 2026-10-03 18:01:47.872 ServerApp notebook_shim | extension was successfully linked.
[1] 2026-10-03 18:01:47.883 ServerApp notebook_shim | extension was successfully loaded.
[1] 2026-10-03 18:01:47.885 ServerApp jupyter_lsp | extension was successfully loaded.
[1] 2026-10-03 18:01:47.885 ServerApp jupyter_server_terminals | extension was successfully loaded.
[1] 2026-10-03 18:01:47.887 LabApp JupyterLab extension loaded from /home/abiska/miniconda3/lib/python3.13/site-packages/jupyterlab
[1] 2026-10-03 18:01:47.887 LabApp JupyterLab application directory is /home/abiska/miniconda3/share/jupyter/lab
[1] 2026-10-03 18:01:47.887 LabApp Extension Manager is 'pypil'.
[1] 2026-10-03 18:01:47.903 ServerApp jupyterlab | extension was successfully loaded.
[1] 2026-10-03 18:01:47.905 ServerApp notebook | extension was successfully loaded.
[1] 2026-10-03 18:01:47.905 ServerApp The port 8888 is already in use, trying another port.
[1] 2026-10-03 18:01:47.905 ServerApp Serving notebooks from local directory: /home/abiska/Documents/Water_Quality_Hazard_Prediction/notebook
[1] 2026-10-03 18:01:47.905 ServerApp Jupyter Server 2.17.0 is running at:
[1] 2026-10-03 18:01:47.905 ServerApp https://localhost:8889/tree?token=6d5a9d5741cff8bb2de15575329f951037a09668532b1699
[1] 2026-10-03 18:01:47.905 ServerApp http://127.0.0.1:8889/tree?token=6d5a9d5741cff8bb2de15575329f951037a09668532b1699
[1] 2026-10-03 18:01:47.905 ServerApp Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[1] 2026-10-03 18:01:47.922 ServerApp

```

Figure 4.7: Docker containers running

4.5 Django Asynchronous View Implementation

The monitor app's `views.py` defines an **asynchronous** Django view that validates the submitted form, forwards it to the FastAPI container using `httpx.AsyncClient()`, persists the reading and its prediction to the `PredictionRecord` model, and renders the result on the same page. Because `httpx.AsyncClient()` performs the outbound call with non-blocking I/O, the Django worker remains free to serve other requests while awaiting the prediction.



Figure 4.8: Source files of the Django portal (monitor app).

4.6 Airflow Retraining DAG

The retraining workflow is defined as an Airflow DAG with two sequential `BashOperator` tasks: `reshape_data`, which runs `data_pipeline.py`, followed by `train_models`, which runs `train.py` against the raw dataset. The DAG runs under Airflow's `LocalExecutor` with a PostgreSQL metadata backend, deployed as an isolated Docker Compose project separate from the main application stack.

```
> water_quality_pipeline_dag.py > ...
from airflow.operators.bash import BashOperator

with DAG(
    dag_id="water_quality_training_pipeline",
    description="Reshape raw sensor data, then train the pH regressor and hazard classifier",
    schedule=None,          # manual trigger only ☐ see docstring above
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=["water-quality", "ml"],
) as dag:

    reshape_data = BashOperator(
        task_id="reshape_data",
        bash_command="cd /opt/airflow/ml_pipeline && python data_pipeline.py",
    )

    train_models = BashOperator(
        task_id="train_models",
        bash_command=(
            "cd /opt/airflow/ml_pipeline && "
            "python train.py --mat-path /opt/airflow/dataset/water_dataset.mat"
        ),
    )

    reshape_data >> train_models
```

Figure 4.9: `airflow-dag.py`

4.7 Drift Monitoring with Evidently AI

The drift-monitoring script loads the processed dataset, splits it into the training split (reference) and test split (current), and runs Evidently's `DataDriftPreset` over the eight model features, producing both a per-feature CSV summary and an HTML drift report.

	A	B	C	D	
1	feature	score	threshold	drifted	
2	temp_mean	0.3905	0.1	True	
3	conductance_mean	0.3506	0.1	True	
4	do_mean	0.3356	0.1	True	
5	ph_max	0.1483	0.1	True	
6	do_max	0.1101	0.1	True	
7	conductance_max	0.1065	0.1	True	
8	conductance_min	0.083	0.1	False	
9	ph_min	0.0555	0.1	False	
10					
11					
12					

Figure 4.10: Evidently script

Chapter 5

Results and Output

5.1 Model Performance Evaluation

5.1.1 Regression: Next-Day pH Forecast

Table 5.1 compares the Random Forest regressor’s performance on the held-out test split before and after Optuna hyperparameter tuning.

Table 5.1: Regression performance, before and after hyperparameter tuning

Configuration	RMSE	MAE	R ²
Before tuning (hand-picked settings)	0.01160	0.00661	0.84476
After Optuna tuning (15 trials)	0.01157	0.00661	0.84564

Tuning produced only a negligible improvement ($\Delta\text{RMSE} \approx 0.00003$), indicating that the untuned Random Forest was already close to this architecture’s performance ceiling on this feature set. This is consistent with Random Forests’ known relative insensitivity to hyperparameter choice compared to other model families.

5.1.2 Classification: Hazard Flag

Table 5.2 summarises the Gradient Boosting classifier’s performance on the held-out test split.

The accuracy figure requires careful interpretation. Because only approximately 4.6% of readings are labelled as hazards, a classifier that always predicted “safe” would already achieve roughly 95% accuracy without learning anything. The model’s 97.4% accuracy is therefore only marginally better than this naive baseline on the accuracy metric alone. **Recall is the more informative metric for this task:** at 0.62, the model correctly identifies only 62% of true hazard readings, meaning approximately one in

Table 5.2: Hazard classification performance (test split)

Metric	Value
Accuracy	0.974
Precision	0.82
Recall	0.62
F1 Score	0.70
ROC-AUC	0.97

three real hazards is missed. For a safety-relevant classifier, this is a meaningful limitation that should be stated plainly rather than obscured behind the headline accuracy figure.

5.1.3 Drift Monitoring Results

Table 5.3 presents the per-feature drift comparison between the training split (reference) and the test split (current), using Evidently AI’s Wasserstein distance metric with a threshold of 0.1.

Table 5.3: Per-feature drift comparison (training split vs. test split)

Feature	Drift Score	Threshold	Drifted?
temp_mean	0.3905	0.1	Yes
conductance_mean	0.3506	0.1	Yes
do_mean	0.3356	0.1	Yes
ph_max	0.1483	0.1	Yes
do_max	0.1101	0.1	Yes
conductance_max	0.1065	0.1	Yes
conductance_min	0.0830	0.1	No
ph_min	0.0555	0.1	No

Six of the eight features (75%) show statistically significant drift. Because the train/test split in this dataset is **temporal** rather than random — the training data covers an earlier contiguous period and the test data covers a later one — this is a genuine finding rather than an artefact of an arbitrary split: it indicates that monitored water conditions at these sites shifted measurably between the two periods, plausibly due to seasonal variation or other environmental change over time.

5.2 MLOps Artefacts and Interfaces

The completed system produces the following observable artefacts, each verified during development:

- **MLflow Tracking UI** – lists the tuned training run with its logged hyperparameters and test-set metrics.

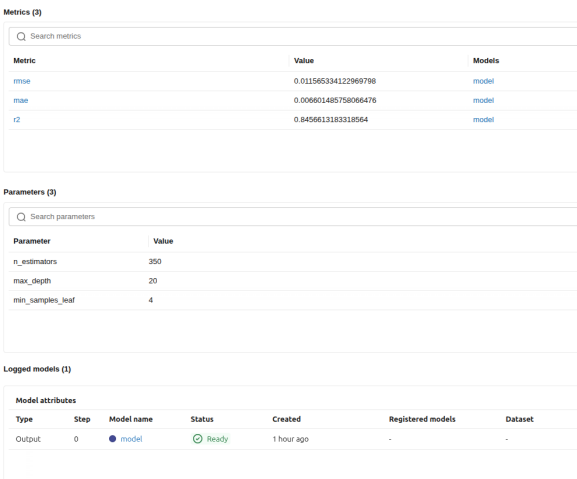


Figure 5.1: Classification metrics on the test split.

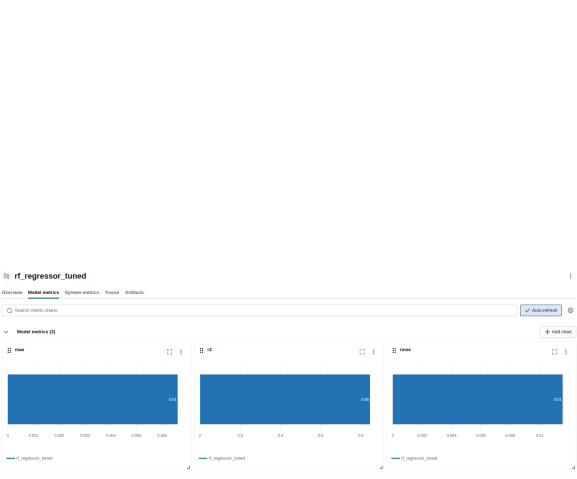


Figure 5.2: Feature importance for the pH regressor.

- **Airflow Web UI** – displays the `water_quality_training_pipeline` DAG, its two-task graph, and the status and logs of each run.

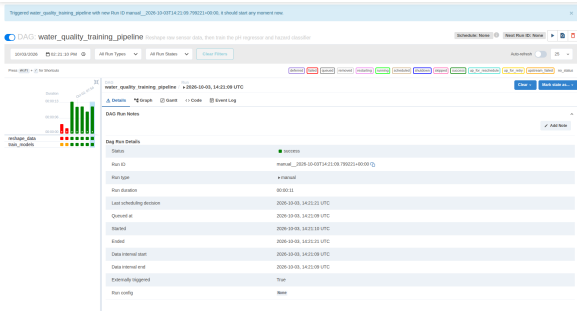


Figure 5.3: Airflow-DAG

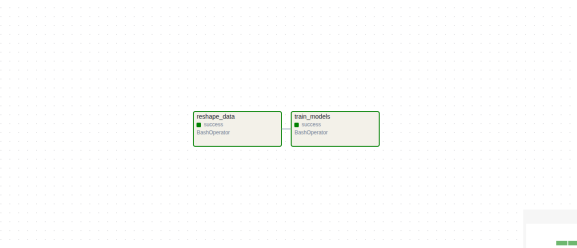


Figure 5.4: Airflow-Graph

- **FastAPI Interactive Documentation** (`/docs`) – an OpenAPI-compliant Swagger UI allowing `/predict` and `/health` to be tested directly from the browser, with schema-validation feedback.

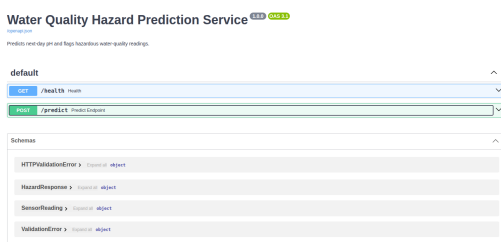


Figure 5.5: FastApi-Dashboard

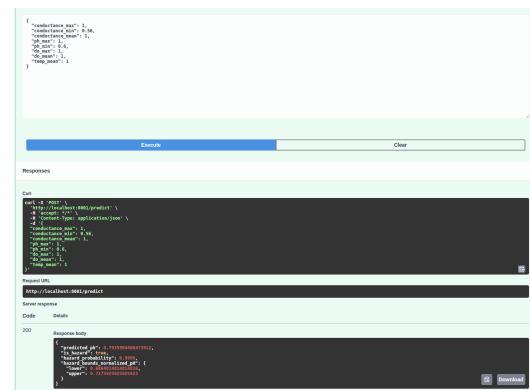


Figure 5.6: FastApi-predict

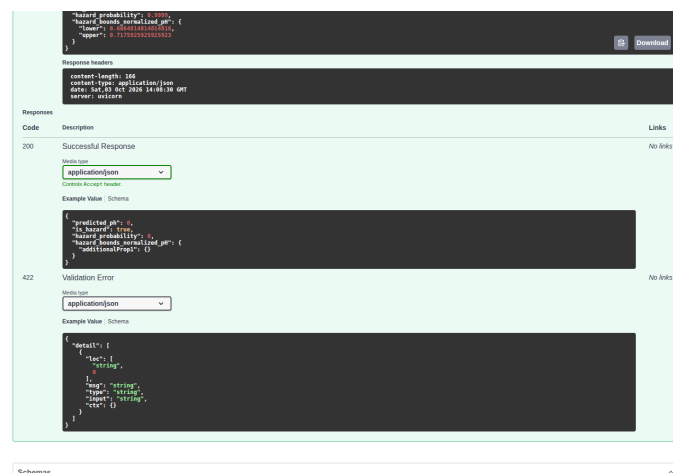


Figure 5.7: FastApi-result

- **Docker Containers** – the FastAPI and Django services start together with a single docker compose up command.

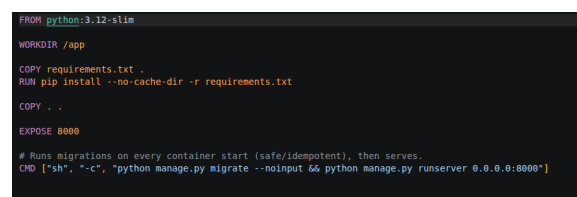


Figure 5.8: Docker file

- **Evidently Drift Report** – an HTML report and CSV summary showing, per feature, the drift score, threshold, and drift decision.



Figure 5.9: Evidently–dashboard.

- **Django Portal** – the results page displays the predicted pH, the hazard verdict, and the hazard probability; submissions and predictions are stored in SQLite and visible on the history page.

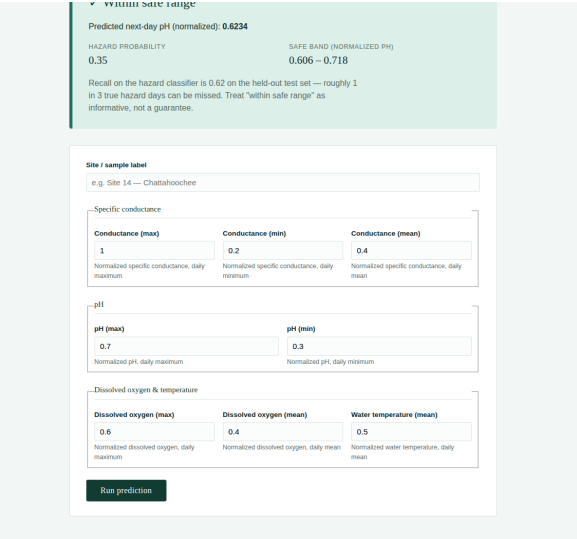


Figure 5.10: Django page with predicted output.

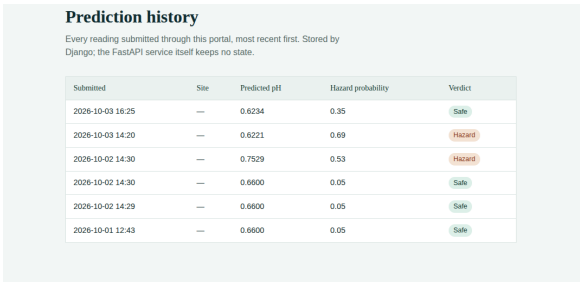


Figure 5.11: Django–History

Chapter 6

Challenges and Limitations

6.1 Technical Hurdles

- **Library API changes across versions.** Several library functions changed their interface between the versions used during development and the versions installed in the final environment — for example, scikit-learn’s `RocCurveDisplay.from_predictions()` moved its `color` argument into a `curve_kwargs` dictionary, and Optuna’s matplotlib plotting functions removed their `ax` parameter in favour of returning their own figure. Each required inspecting the new function signature and updating the corresponding code.
- **Container file-permission mismatch.** The first Airflow DAG run failed with a `PermissionError` when writing the reshaped dataset to a mounted volume. The cause was a mismatch between the container’s configured `AIRFLOW_UID` and the host user’s actual UID, which denied write access to the bind-mounted directory. Setting `AIRFLOW_UID` to the host user’s real UID resolved the issue.
- **Interpreting an imbalanced classification metric.** With hazards representing only 4.6% of readings, the headline accuracy score was initially misleading. Precision, recall, and ROC-AUC were used instead to give an honest picture of classifier performance, as discussed in Section 5.1.3.
- **Orchestrating multiple tools together.** Airflow, MLflow, Docker, and Evidently each have independent configuration and dependency requirements. Running Airflow as an isolated Docker Compose project with its own executor and metadata database, separate from the main application stack, avoided port and resource conflicts.

6.2 System Constraints

- **Hazard label is a statistical proxy.** Because the dataset's original normalisation scaler is not recoverable, the hazard label is defined via an IQR outlier rule on normalised pH rather than a true regulatory threshold such as the EPA's pH 6.5–8.5 potable-water range. This is an intentional, documented simplification, not an oversight.
- **Single-node deployment.** All components currently run on a single machine via Docker Compose. A production-grade deployment would require an orchestrator such as Kubernetes, a reverse proxy, and a managed relational database in place of SQLite.
- **No ground-truth feedback loop.** Drift monitoring can detect when incoming data differs from the training distribution, but cannot measure live model accuracy, since true outcomes are not collected after a prediction is made.
- **Manually triggered retraining.** The Airflow DAG is triggered on demand rather than on an automatic schedule or in response to detected drift, which is a reasonable simplification for a project evaluated once but would need to change for a genuinely production system.

Chapter 7

Conclusion

The goal of this project is to showcase the design and implementation of an end-to-end MLOps pipeline using two complementary models: a pH regressor, and a hazard classifier, trained on real water-quality sensor data from the United States Geological Survey (USGS) from the state of Georgia, USA. While this systematic tuning of hyperparameters was performed using Optuna, tracking was done using MLflow to ensure reproducibility. The process of retraining the models is automated in Apache Airflow, the trained models are served via a Pydantic validated FastAPI microservice in Docker, and a Django portal with a SQLite database offers the user facing interface.

These components solve the four issues mentioned in the introduction: Tight coupling and latency, lack of boundary validation, untracked model development, and silent model degradation. This pattern, which is the representation of a thin web application and the inference logic delegated to an independently deployable, automatically retrained and continuously monitored model service, extends to a wider range of AI integrated systems beyond the water quality case.

Chapter 8

Future Scope

Several enhancements have been identified for extending this project:

- **Automated, drift-triggered retraining.** Replace the manually triggered DAG with a sensor-based trigger that initiates retraining automatically when Evidently detects significant drift, with new model versions only promoted if they outperform the currently deployed model on a validation set.
- **A recoverable hazard threshold.** Source a version of this dataset (or a comparable one) with an unnormalised pH scale, or incorporate a real regulatory threshold such as the EPA's pH 6.5–8.5 potable-water range, to replace the current IQR-based statistical proxy with a genuine safety threshold.
- **Full containerised orchestration.** Extend the current Docker Compose setup toward a Kubernetes deployment with a managed PostgreSQL database in place of SQLite, for production-scale concurrent use.
- **Ground-truth feedback and live performance monitoring.** Collect verified outcomes for submitted samples so that live model accuracy, not just input drift, can be tracked over time.
- **Geospatial visualisation.** Display submitted readings and their hazard status on an interactive map to identify regional hazard patterns across monitoring sites.

Chapter 9

References

- [1] UCI Machine Learning Repository. (2022). *Water Quality Prediction* [Dataset]. Retrieved from <https://archive.ics.uci.edu/dataset/733/water+quality+prediction-1>
- [2] Zhao, L., Gkountouna, O., & Pfoser, D. (2019). Spatial Auto-regressive Dependency Interpretable Learning Based on Spatial Topological Constraints. *ACM Transactions on Spatial Algorithms and Systems*, 5(3), Article 19. <https://doi.org/10.1145/3339823>
- [3] Django Software Foundation. *Django Documentation*. Retrieved from <https://docs.djangoproject.com/>
- [4] Ramírez, S. *FastAPI Documentation*. Retrieved from <https://fastapi.tiangolo.com/>
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [6] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. Retrieved from <https://optuna.org/>
- [7] MLflow Project. *MLflow Documentation*. Retrieved from <https://mlflow.org/docs/latest/>
- [8] Apache Software Foundation. *Apache Airflow Documentation*. Retrieved from <https://airflow.apache.org/docs/>
- [9] Evidently AI. *Evidently Documentation*. Retrieved from <https://docs.evidentlyai.com/>
- [10] Docker Inc. *Docker Documentation*. Retrieved from <https://docs.docker.com/>

-
- [11] Pydantic Developers. *Pydantic Documentation*. Retrieved from <https://docs.pydantic.dev/>
 - [12] HTTPX Developers. *HTTPX Documentation*. Retrieved from <https://www.python-httpx.org/>
 - [13] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.

Chapter 10

Appendices

Appendix A: Terminal Setup and Deployment Commands

The following commands document the setup procedure used for this project: creating virtual environments, running the notebook, starting the containerised FastAPI and Django services, and running the Airflow retraining pipeline.

```
abiska@abiska-IdeaPad-Slim-5-16IMH9:~$ find ~ -iname "Water_Quality_Hazard_Prediction*" 2>/dev/null
abiska@abiska-IdeaPad-Slim-5-16IMH9:~$ find ~/Downloads -iname "Water_Quality_Hazard_Prediction*" 2>/dev/null
abiska@abiska-IdeaPad-Slim-5-16IMH9:~$ find ~/Documents -iname "Water_Quality_Hazard_Prediction*" 2>/dev/null
/home/abiska/Documents/Water_Quality_Hazard_Prediction.zip
abiska@abiska-IdeaPad-Slim-5-16IMH9:~$ cd ~/Documents
unzip Water_Quality_Hazard_Prediction.zip
grep loadmat Water_Quality_Hazard_Prediction/notebook/water_quality_model_training.ipynb
Archive: Water_Quality_Hazard_Prediction.zip
  creating: Water_Quality_Hazard_Prediction/
  creating: Water_Quality_Hazard_Prediction/django_portal/
 extracting: Water_Quality_Hazard_Prediction/django_portal/requirements.txt
  creating: Water_Quality_Hazard_Prediction/django_portal/water_portal/
 extracting: Water_Quality_Hazard_Prediction/django_portal/water_portal/__init__.py
 inflating: Water_Quality_Hazard_Prediction/django_portal/water_portal/settings.py
 inflating: Water_Quality_Hazard_Prediction/django_portal/water_portal/asgi.py
 inflating: Water_Quality_Hazard_Prediction/django_portal/water_portal/urls.py
 inflating: Water_Quality_Hazard_Prediction/django_portal/water_portal/wsgi.py
 inflating: Water_Quality_Hazard_Prediction/django_portal/manage.py
  creating: Water_Quality_Hazard_Prediction/django_portal/monitor/
 extracting: Water_Quality_Hazard_Prediction/django_portal/monitor/__init__.py
  creating: Water_Quality_Hazard_Prediction/django_portal/monitor/static/
  creating: Water_Quality_Hazard_Prediction/django_portal/monitor/static/monitor/
  creating: Water_Quality_Hazard_Prediction/django_portal/monitor/static/monitor/css/
 inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/static/monitor/css/style.css
  creating: Water_Quality_Hazard_Prediction/django_portal/monitor/migrations/
 extracting: Water_Quality_Hazard_Prediction/django_portal/monitor/migrations/__init__.py
 inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/migrations/0001_initial.py
  creating: Water_Quality_Hazard_Prediction/django_portal/monitor/templates/
  creating: Water_Quality_Hazard_Prediction/django_portal/monitor/templates/monitor/
 inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/templates/monitor/predict_form.html
 inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/templates/monitor/history.html
 inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/templates/monitor/base.html
 inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/admin.py
 inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/tests.py
```

```
inflating: Water_Quality_Hazard_Prediction/django_portal/wsgi.py
inflating: Water_Quality_Hazard_Prediction/django_portal/manage.py
creating: Water_Quality_Hazard_Prediction/django_portal/monitor/
extracting: Water_Quality_Hazard_Prediction/django_portal/monitor/__init__.py
creating: Water_Quality_Hazard_Prediction/django_portal/monitor/static/
creating: Water_Quality_Hazard_Prediction/django_portal/monitor/static/monitor/
creating: Water_Quality_Hazard_Prediction/django_portal/monitor/static/monitor/css/
inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/static/monitor/css/style.css
creating: Water_Quality_Hazard_Prediction/django_portal/monitor/migrations/
extracting: Water_Quality_Hazard_Prediction/django_portal/monitor/migrations/__init__.py
inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/migrations/0001_initial.py
creating: Water_Quality_Hazard_Prediction/django_portal/monitor/templates/
creating: Water_Quality_Hazard_Prediction/django_portal/monitor/templates/monitor/
inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/templates/monitor/predict_form.html
inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/templates/monitor/history.html
inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/templates/monitor/base.html
inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/admin.py
inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/tests.py
inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/models.py
inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/forms.py
inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/views.py
inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/apps.py
inflating: Water_Quality_Hazard_Prediction/django_portal/monitor/urls.py
creating: Water_Quality_Hazard_Prediction/report/
inflating: Water_Quality_Hazard_Prediction/report/PLACEHOLDER.txt
inflating: Water_Quality_Hazard_Prediction/README.md
creating: Water_Quality_Hazard_Prediction/ml_pipeline/
inflating: Water_Quality_Hazard_Prediction/ml_pipeline/requirements.txt
inflating: Water_Quality_Hazard_Prediction/ml_pipeline/fastapi_service.py
inflating: Water_Quality_Hazard_Prediction/ml_pipeline/README.md
inflating: Water_Quality_Hazard_Prediction/ml_pipeline/data_pipeline.py
creating: Water_Quality_Hazard_Prediction/ml_pipeline/models/
inflating: Water_Quality_Hazard_Prediction/ml_pipeline/models/ph_regressor.joblib
inflating: Water_Quality_Hazard_Prediction/ml_pipeline/models/hazard_classifier.joblib
inflating: Water_Quality_Hazard_Prediction/ml_pipeline/models/metadata.json
creating: Water_Quality_Hazard_Prediction/ml_pipeline/data/
inflating: Water_Quality_Hazard_Prediction/ml_pipeline/data/water_quality_processed.csv
inflating: Water_Quality_Hazard_Prediction/ml_pipeline/predict.py
inflating: Water_Quality_Hazard_Prediction/ml_pipeline/train.py
creating: Water_Quality_Hazard_Prediction/notebook/
inflating: Water_Quality_Hazard_Prediction/notebook/water_quality_model_training.ipynb
creating: Water_Quality_Hazard_Prediction/dataset/
inflating: Water_Quality_Hazard_Prediction/dataset/water_dataset.mat
inflating: Water_Quality_Hazard_Prediction/dataset/original_dataset_README.docx
inflating: Water_Quality_Hazard_Prediction/dataset/water_quality_processed.csv
"mat = sio.loadmat("../dataset/water_dataset.mat")\n",
```

```

abiska@abiska-IdeaPad-Slim-5-16IMH9:~/Documents$ cd ~/Documents/Water_Quality_Hazard_Prediction/ml_pipeline
python3 -m venv venv
source venv/bin/activate
(venv) abiska@abiska-IdeaPad-Slim-5-16IMH9:~/Documents/Water_Quality_Hazard_Prediction/ml_pipeline$ which python3
/home/abiska/Documents/Water_Quality_Hazard_Prediction/ml_pipeline/venv/bin/python3
(venv) abiska@abiska-IdeaPad-Slim-5-16IMH9:~/Documents/Water_Quality_Hazard_Prediction/ml_pipeline$ pip install -r requirements.txt
pip install jupyter matplotlib seaborn nbformat
Collecting pandas>=2.0 (from -r requirements.txt (line 1))
  Using cached pandas-3.0.6-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
Collecting numpy>=1.24 (from -r requirements.txt (line 2))
  Using cached numpy-2.5.3-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting scipy>=1.10 (from -r requirements.txt (line 3))
  Using cached scipy-1.18.1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (62 kB)
Collecting scikit-learn>=1.3 (from -r requirements.txt (line 4))
  Using cached scikit_learn-1.9.1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (9.3 kB)
Collecting joblib>=1.3 (from -r requirements.txt (line 5))
  Using cached joblib-1.6.0-py3-none-any.whl.metadata (6.5 kB)
Collecting fastapi>=0.110 (from -r requirements.txt (line 6))
  Using cached fastapi-0.142.1-py3-none-any.whl.metadata (27 kB)
Collecting uvicorn>=0.29 (from uvicorn[standard]>=0.29->-r requirements.txt (line 7))
  Using cached uvicorn-0.54.0-py3-none-any.whl.metadata (6.6 kB)
Collecting pydantic>=2.0 (from -r requirements.txt (line 8))
  Using cached pydantic-2.13.5-py3-none-any.whl.metadata (110 kB)
Collecting python-dateutil>=2.8.2 (from pandas>=2.0->-r requirements.txt (line 1))
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting narwhals>=2.0.1 (from scikit-learn>=1.3->-r requirements.txt (line 4))
  Using cached narwhals-2.26.0-py3-none-any.whl.metadata (15 kB)
Collecting threadpoolctl>=3.5.0 (from scikit-learn>=1.3->-r requirements.txt (line 4))
  Using cached threadpoolctl-3.7.0-py3-none-any.whl.metadata (24 kB)
Collecting cloudpickle>=3.0 (from joblib>=1.3->-r requirements.txt (line 5))
  Using cached cloudpickle-3.1.2-py3-none-any.whl.metadata (7.1 kB)
Collecting starlette>=0.46.0 (from fastapi>=0.110->-r requirements.txt (line 6))
  Using cached starlette-1.7.0-py3-none-any.whl.metadata (6.6 kB)
Collecting typing-extensions>=4.8.0 (from fastapi>=0.110->-r requirements.txt (line 6))
  Using cached typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting typing-inspection>=0.4.2 (from fastapi>=0.110->-r requirements.txt (line 6))
  Using cached typing_inspection-0.4.4-py3-none-any.whl.metadata (2.6 kB)
Collecting annotated-doc>=0.0.2 (from fastapi>=0.110->-r requirements.txt (line 6))
  Using cached annotated_doc-0.0.5-py3-none-any.whl.metadata (6.5 kB)
Collecting opentelemetry-api>=1.44.0 (from fastapi>=0.110->-r requirements.txt (line 6))
  Using cached opentelemetry_api-1.45.0-py3-none-any.whl.metadata (1.4 kB)
Collecting click>=7.0 (from uvicorn>=0.29->uvicorn[standard]>=0.29->-r requirements.txt (line 7))
  Using cached click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting h11>=0.8 (from uvicorn>=0.29->uvicorn[standard]>=0.29->-r requirements.txt (line 7))
  Using cached h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Collecting annotated-types>=0.6.0 (from pydantic>=2.0->-r requirements.txt (line 8))

```

```

Using cached python_json_logger-4.2.0-py3-none-any.whl (14 kB)
Using cached requests-2.34.2-py3-none-any.whl (73 kB)
Downloading charset_normalizer-3.5.2-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (254 kB)
Using cached urllib3-2.8.0-py3-none-any.whl (135 kB)
Using cached certifi-2026.7.22-py3-none-any.whl (136 kB)
Using cached rfc3986_validator-0.1.1-py2.py3-none-any.whl (4.2 kB)
Using cached rfc3987_syntax-1.1.0-py3-none-any.whl (8.0 kB)
Using cached lark-1.3.1-py3-none-any.whl (113 kB)
Using cached send2trash-2.1.0-py3-none-any.whl (17 kB)
Using cached terminado-0.18.1-py3-none-any.whl (14 kB)
Using cached tinycss2-1.5.1-py3-none-any.whl (28 kB)
Using cached webcolors-25.10.0-py3-none-any.whl (14 kB)
Using cached webencodings-0.6.1-py3-none-any.whl (8.7 kB)
Using cached websocket-client-1.9.2-py3-none-any.whl (95 kB)
Using cached argon2_cfapi_bindings-26.1.0-cp318-abt3-manylinux_2_26_x86_64.manylinux_2_28_x86_64.whl (26 kB)
Using cached cffi-2.1.1-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (221 kB)
Using cached beautifulsoup4-4.15.0-py3-none-any.whl (109 kB)
Using cached soupsieve-2.10-py3-none-any.whl (38 kB)
Using cached defusedxml-0.7.1-py2.py3-none-any.whl (25 kB)
Using cached fqdn-1.6.0-py3-none-any.whl (12 kB)
Using cached isoduration-20.11.0-py3-none-any.whl (11 kB)
Using cached arrow-1.4.0-py3-none-any.whl (68 kB)
Using cached jupyterlab_pygments-0.3.0-py3-none-any.whl (15 kB)
Using cached notebook-7.6.3-py3-none-any.whl (5.5 MB)
Using cached pure-eval-0.2.4-py3-none-any.whl (11 kB)
Using cached pyparser-3.0-py3-none-any.whl (48 kB)
Using cached rfc3339_validator-0.1.4-py2.py3-none-any.whl (3.5 kB)
Using cached tzdata-2026.4-py2.py3-none-any.whl (347 kB)
Using cached uri_template-1.3.0-py3-none-any.whl (11 kB)
Installing collected packages: pure-eval, ptyprocess, widgetsnbextension, websocket-client, webencodings, webcolors, wcwidth, urllib3, uri-template, tzdata, traitlets, tornado, soupsie
ve, send2trash, rpdps-py, rfc3986-validator, rfc3339-validator, pyzmq, python-json-logger, pyqrcode, python-cv2, pygments, pyparser, psutil, prometheus-client, platformdirs, pillow, pexpect, pars
o, pandocfilters, packaging, nest-asyncio2, mistune, MarkupSafe, lark, kiwisolver, jupyterlab_widgets, jupyterlab-pygments, jsonpointer, json5, fqdn, fonttools, fastjsonschema, executin
g, defusedxml, debugpy, cycloper, contourpy, comm, charset_normalizer, certifi, babel, attrs, async-lru, asttokens, tinycss2, terminado, stack_data, rfc3987-syntax, requests, referencin
g, prompt_toolkit, matplotlib-inline, matplotlib, jupyter-core, jinja2, jedi, ipython-pygments-lexers, httpcore, cffi, bleach, beautifulsoup4, arrow, seaborn, jupyter-server-terminals
, jupyter-client, jupyter-builder, jsonschema-specifications, isoduration, ipython, httpx, argon2-cffi-bindings, jsonschema, iawidgets, tpykernel, argon2-cffi, nbformat, jupyter-consol
e, nbclient, jupyter-events, nbconvert, jupyter-server, notebook-shim, jupyterlab-server, jupyter-lsp, jupyterlab, notebook, jupyter
Successfully installed MarkupSafe-3.0.3 argon2-cffi-25.1.0 argon2-cffi-bindings-26.1.0 arrow-1.4.0 asttokens-3.0.2 async-lru-2.3.0 attrs-26.1.0 babel-2.18.0 beautifulsoup4-4.15.0 bleac
h-6.4.0 certifi-2026.7.22 cffi-2.1.1 charset_normalizer-3.5.2 comm-0.2.3 contourpy-1.4.0 cycloper-1.8.22 defusedxml-0.7.1 executing-2.2.1 fastjsonschema-2.22.2 fonttools-4
.66.1 fqdn-1.6.0 httpcore-1.0.9 httpx-0.28.1 ipykernel-7.4.0 ipython-9.17.1 ipython-pygments-lexers-1.1.1 ipywidgets-8.1.9 isoduration-20.11.0 jedi-0.20.0 jinja2-3.1.6 json5-0.15.0 jso
npointer-3.1.1 jsonschema-4.26.0 jsonschema-specifications-2025.9.1 jupyter-1.1.1 jupyter-builder-1.2.3 jupyter-client-8.10.0 jupyter-console-6.6.3 jupyter-core-5.9.1 jupyter-events-0
.12.1 jupyter-lsp-2.3.1 jupyter-server-2.21.1 jupyter-server-terminals-0.5.4 jupyterlab-4.6.4 jupyterlab-4.6.4 jupyterlab-pygments-0.3.0 jupyterlab-server-2.28.1 jupyterlab_widgets-3.0.17 kiwisolver-1.5
.1 lark-1.3.1 matplotlib-3.11.2 matplotlib-inline-0.2.2 mistune-3.3.4 nbclient-0.11.0 nbconvert-7.17.1 nbformat-5.11.1 nest-asyncio2-1.7.3 notebook-7.6.3 notebook-shim-0.2.4 packaging-
26.3 pandocfilters-1.5.1 pako-0.8.7 pexpect-4.9.0 pillow-12.3.0 platformdirs-4.12.2 prometheus-client-0.26.0 prompt_toolkit-3.0.53 psutil-7.2.2 ptyprocess-0.7.0 pure-eval-0.2.4 pypar
ser-3.0 pygments-2.21.0 pyqrcode-3.3.2 python-json-logger-4.2.0 pyzmq-27.2.0 referencing-0.37.0 requests-2.34.2 rfc3339-validator-0.1.4 rfc3986-validator-0.1.1 rfc3987-syntax-1.1.0 rp
ds-py-2026.6.3 seaborn-0.13.2 send2trash-2.1.0 soupsieve-2.10 stack_data-0.6.3 terminado-0.18.1 tinycss2-1.5.1 tornado-6.5.10 traitlets-5.16.1 tzdata-2026.4 uri-template-1.3.0 urllib3-
2.8.0 wcwidth-0.9.1 webcolors-25.10.0 webencodings-0.6.1 websocket-client-1.9.2 widgetsnbextension-4.0.16
[notice] A new release of pip is available: 25.3 -> 26.2.1
[notice] To update, run: pip install --upgrade pip

```

```
abiska@abiska-IdeaPad-Slim-5-16IMH9:~/Documents/Water_Quality_Hazard_Prediction/notebook$ pip install optuna
Requirement already satisfied: optuna in /home/abiska/miniconda3/lib/python3.13/site-packages (4.9.0)
Requirement already satisfied: alembic>=1.5.0 in /home/abiska/miniconda3/lib/python3.13/site-packages (from optuna) (1.18.4)
Requirement already satisfied: colorlog in /home/abiska/miniconda3/lib/python3.13/site-packages (from optuna) (6.10.1)
Requirement already satisfied: numpy in /home/abiska/miniconda3/lib/python3.13/site-packages (from optuna) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /home/abiska/miniconda3/lib/python3.13/site-packages (from optuna) (25.0)
Requirement already satisfied: sqlalchemy>=1.4.2 in /home/abiska/miniconda3/lib/python3.13/site-packages (from optuna) (2.0.49)
Requirement already satisfied: tqdm in /home/abiska/miniconda3/lib/python3.13/site-packages (from optuna) (4.67.3)
Requirement already satisfied: PyYAML in /home/abiska/miniconda3/lib/python3.13/site-packages (from optuna) (6.0.3)
Requirement already satisfied: Mako in /home/abiska/miniconda3/lib/python3.13/site-packages (from alembic>=1.5.0->optuna) (1.3.12)
Requirement already satisfied: typing-extensions>=4.12 in /home/abiska/miniconda3/lib/python3.13/site-packages (from alembic>=1.5.0->optuna) (4.15.0)
Requirement already satisfied: greenlet>=1 in /home/abiska/miniconda3/lib/python3.13/site-packages (from sqlalchemy>=1.4.2->optuna) (3.4.0)
Requirement already satisfied: MarkupSafe>=0.9.2 in /home/abiska/miniconda3/lib/python3.13/site-packages (from Mako->alembic>=1.5.0->optuna) (3.0.3)

[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
```